

SCHWAB ASSIGNMENT

Agentic SDLC System

An agentic, multi-model pipeline that transforms a plain-English requirement into a full software deliverable — plan, architecture, tests, generated code, docs, and a human-approved release decision.

Spring AI 1.0	LangGraph4j 1.8	Ollama LLM	Java 21	8 Agents
---------------	-----------------	------------	---------	----------



1. System Overview

You submit a plain-English request — "Build a URL shortener" — and the system autonomously runs through every phase of the software development lifecycle, producing engineering artifacts at each stage before pausing for a human decision on whether to ship.



Architecture Layers

- 1 HTTP API Layer**
SdlcController + UriShortenerController — REST endpoints. Two domains: URL shortener (the software being built) and SDLC orchestration. Separate controllers, same server.
- 2 Orchestration Graph**
SdlcOrchestrator (LangGraph4j) — stateful directed graph. Nodes are agents. Parallel branches fan out via CompletableFuture, rejoin before implementation.
- 3 Agent Layer**
8 specialized agents. Each calls the LLM gateway with its agentKey and parses the response into a typed record (e.g. RequirementsOutput). Writes results into SdlcState.
- 4 LLM Gateway**
OllamaLlmGateway routes each agent to its model (llama3.1:8b or qwen2.5-coder:7b). Health-probes Ollama; falls back to OfflineLlmGateway when unreachable.
- 5 Governance Layer**
Audit log (hash-chained), PolicyEngine, HumanGate (pauses graph), ReliabilityMetrics records latency per agent.
- 6 Persistence**
PausedRunRepository via JdbcTemplate + H2. Flyway migrations V1-V3. Paused runs restored on ApplicationReadyEvent.

2. Execution Flow

From API call to human decision. The graph is stateful — every node reads and writes SdlcState. Rejection loops back to Planning.

Step 1 — POST /api/v1/sdlc/runs

Client sends {"requirement": "Build a URL shortener"}. SdlcController creates SdlcState (UUID run ID, status=running), starts graph on background thread, immediately returns 202 Accepted with runId.

Step 2 — Requirements Agent

Prompts llama3.1:8b. Parses JSON into RequirementsOutput{functional[], nonFunctional[], constraints[]}. Stores in SdlcState. Adds audit entry.

Step 3 — Planning Agent

Receives requirements text. Returns PlanningOutput{phases[], milestones[], resourcePlan}. Adds audit entry.

Step 4 — Parallel Branches — CompletableFuture.allOf()

Three agents run concurrently on a 3-thread pool: Architecture (HLD, components, tech stack), Risk Assessment (risks[], mitigations[]), Test Strategy (unit/integration/e2e plans). Wall time = max of three, not sum.

Step 5 — Codebase Analysis Agent

Runs after all branches rejoin. Identifies impacted modules, estimates complexity. Feeds context into Implementation.

Step 6 — Implementation Agent

Calls qwen2.5-coder:7b — the dedicated coding model. Generates Java source (controller, service, repository, tests). Output stored as raw string in SdlcState.

Step 7 — Documentation Agent

Produces DocumentationOutput{readme, apiDocs, runbook, changelog} using llama3.1:8b.

Step 8 — Code Validation Node

Extracts Java from implementation output. Compiles via javax.tools.JavaCompiler. Loads via URLClassLoader, runs test* methods by reflection. Records ValidationResult{passed, compilerOutput, testResults[], durationMs}.

Step 9 — Human Gate — PAUSE

PolicyEngine checks risk thresholds and validation status. Graph pauses. PausedRunRepository serializes SdlcState to H2. Audit: AWAITING_HUMAN_DECISION.

Step 10 — POST /runs/{id}/decisions

Authorized approver submits decision. Controller checks approver whitelist → 403 if not authorized. Approve: status=completed. Reject: loops back to Planning.

3. Agents & Models

Each agent is a Spring @Component implementing `run(SdmcState state)`. Agents call `LlmGateway.call(agentKey, prompt)` — they never reference Ollama directly.

Agent	Agent Key	Model	Output Type	Role
RequirementsAgent	requirements_analysis	llama3.1:8b	RequirementsOutput	Extracts functional/non-functional requirements
PlanningAgent	planning	llama3.1:8b	PlanningOutput	Creates phased delivery plan and milestones
ArchitectureAgent	architecture_design	llama3.1:8b	ArchitectureOutput	Designs HLD, components, data model, API contracts
RiskAssessmentAgent	risk_assessment	llama3.1:8b	RiskOutput	Identifies risks with severity and mitigations
TestStrategyAgent	test_strategy	llama3.1:8b	TestStrategyOutput	Plans unit, integration, and E2E test approaches
CodebaseAnalysisAgent	codebase_analysis	llama3.1:8b	CodebaseOutput	Identifies impacted modules, estimates complexity
ImplementationAgent	implementation	qwen2.5-coder:7b	String (raw Java)	Generates working Java source code and unit tests
DocumentationAgent	documentation	llama3.1:8b	DocumentationOutput	Writes README, API docs, runbook, changelog

Parallel Execution Code

```
// SdmcOrchestrator.java – parallel branch
ExecutorService pool = Executors.newFixedThreadPool(3);

CompletableFuture<Void> archFuture = CompletableFuture.runAsync(
    () -> state.setArchitectureOutput(architectureAgent.run(state)), pool);
CompletableFuture<Void> riskFuture = CompletableFuture.runAsync(
    () -> state.setRiskOutput(riskAgent.run(state)), pool);
CompletableFuture<Void> testFuture = CompletableFuture.runAsync(
    () -> state.setTestStrategyOutput(testAgent.run(state)), pool);

// Wall time = max(arch, risk, test) – not sum
CompletableFuture.allOf(archFuture, riskFuture, testFuture)
    .get(120, TimeUnit.SECONDS);
```

4. LLM Gateway

A clean interface hides all model-routing and fallback logic from agents. Every agent only calls `gateway.call(agentKey, prompt)`.

```
public interface LlmGateway {
    String call(String agentKey, String prompt);
    boolean isAvailable();
}
```

PRIMARY

OllamaLlmGateway

Calls Ollama via Spring AI's `OllamaChatModel`. Selects model per-request via `OllamaOptions`. Health-probes before each call. Tracks per-agent failure counts. Falls back on any exception.

FALLBACK

OfflineLlmGateway

Returns deterministic, pre-baked JSON fixtures keyed by `agentKey`. No network. Used when Ollama is unreachable or any Ollama call fails. Guarantees the graph always completes.

Output Cleaning Pipeline

LLMs return messy text. The gateway strips noise before passing responses to agents.

Step 1 — Strip blocks

`gwen3` / `deepseek-r1` emit chain-of-thought inside `...` tags. Removed via regex before any other processing.

Step 2 — Extract JSON from markdown fences

Models often wrap JSON in ````json ... ````. The gateway extracts the inner content. Uses `lastIndexOf("```")` for the closing fence to handle fences-within-fences.

Step 3 — Return clean string

Agents receive raw JSON or clean prose. They parse it with Jackson's `BeanOutputConverter` to get their typed output record.

Health Probe (cached 30 s)

```
// OllamaHealthProbe - probes GET /api/tags, caches 30s
private CachedResult probe() {
    CachedResult cached = cache.get();
    if (cached != null &&
        Duration.between(cached.fetchedAt(), Instant.now())
            .getSeconds() < cacheTtlSeconds) {
        return cached; // cache hit - no network call
    }
    CachedResult fresh = fetchFromOllama();
    cache.set(fresh);
    return fresh;
}
```

5. Governance & Safety Controls

Autonomous agents must earn the right to ship. Every decision is recorded, every approval is verified, and the final release requires a human.

Human Gate

PAUSED After all 8 agents complete and code is validated, the LangGraph4j state machine transitions to status=paused and stops. The run persists to H2. Resumption requires an explicit POST /runs/{id}/decisions from an authorized approver.

Approver Whitelist (HTTP 403)

```
// SdlcController.java
List<String> approvers = sdlcProperties.getApprovers();
if (!approvers.contains(req.getApprover())) {
    return ResponseEntity.status(403)
        .body(Map.of("error", "Approver not authorized",
            "approver", req.getApprover()));
}
```

Tamper-Evident Audit Log

Every state transition adds an entry. Each entry includes a SHA-256 hash chained with the previous entry's hash.

```
{
  "timestamp": "2026-09-28T15:23:11Z",
  "event": "AGENT_COMPLETE",
  "actor": "agent:implementation/llm[ollama:qwen2.5-coder:7b]",
  "detail": "Generated 234 lines of Java",
  "prevHash": "a3f9...",
  "hash": "7c12..."
}
```

Code Validation Node

After implementation, generated Java is compiled in a sandbox and test methods are executed via reflection. The graph always continues — passed=false is recorded, never thrown.

```
// CodeValidationNode.java
JavaCompiler compiler = ToolProvider.getSystemJavaCompiler();
DiagnosticCollector<JavaFileObject> diags = new DiagnosticCollector<>();
// write generated source to temp dir, compile
JavaCompiler.CompilationTask task = compiler.getTask(null, fm, diags, null, null, units);
boolean ok = task.call();
// load class, reflect on test* methods, run them
Class<?> cls = new URLClassLoader(new URL[]{tempDir.toURI().toURL()})
    .loadClass("GeneratedTest");
for (Method m : cls.getMethods()) {
    if (m.getName().startsWith("test")) m.invoke(null);
}
```

State Persistence (H2 + Flyway)

```
-- V3__paused_runs.sql
CREATE TABLE IF NOT EXISTS paused_runs (
    run_id      VARCHAR(36) NOT NULL PRIMARY KEY,
    state_json  TEXT        NOT NULL,
    gate        VARCHAR(100),
    paused_at   TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

On `ApplicationReadyEvent`, `PausedRunRepository` restores all persisted runs back into the in-memory run map. A server restart cannot lose a pending approval.

6. REST API Reference

URL Shortener Domain

Method	Path	Status	Description
POST	/api/v1/shorten	201	Create short URL. Body: {originalUrl, customAlias}
GET	/[{code}]	302	Redirect to original URL. Records click event.
GET	/api/v1/urls	200	List all active URLs.
GET	/api/v1/urls/{code}/analytics	200	Click count + event timestamps.
DELETE	/api/v1/urls/{code}	204	Deactivate (soft delete).

SDLC Orchestration Domain

Method	Path	Status	Description
POST	/api/v1/sdlc/runs	202	Start new SDLC run. Returns {runId} immediately.
GET	/api/v1/sdlc/runs/{id}	200	Full SdlcState including all agent outputs.
POST	/api/v1/sdlc/runs/{id}/decisions	200/403	Approve or reject. 403 if approver not whitelisted.
GET	/api/v1/sdlc/runs	200	List all paused and running runs.
GET	/api/v1/sdlc/runs/{id}/audit	200	Hash-chained audit log entries.
GET	/api/v1/sdlc/runs/{id}/policy	200	PolicyEngine result: passed/failed + violations.
GET	/api/v1/sdlc/agents	200	AgentModelRoster: which model each agent uses.
GET	/api/v1/sdlc/metrics	200	Per-agent latency, success rate, p95.

Key SdlcState Response Shape

```
// GET /api/v1/sdlc/runs/{id}
{
  "runId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "status": "paused",           // running | paused | completed | failed
  "requirement": "Build a URL shortener...",
  "requirementsOutput": { "functional": [...], ... },
  "planningOutput": { "phases": [...], ... },
  "architectureOutput": { ... },
  "riskOutput": { "risks": [...] },
  "testStrategyOutput": { ... },
  "implementationOutput": "// Generated Java source...",
  "documentationOutput": { "readme": "...", ... },
  "validationResult": {
    "passed": true,
    "compilerOutput": "",
    "testResults": ["testShortenUrl: PASS"],
    "durationMs": 342
  },
  "gate": "HUMAN_REVIEW"
}
```

7. Infrastructure & Technology Stack

INFO

Runtime

Java 21 · Spring Boot 3.3 · Spring AI 1.0.0 ·
LangGraph4j 1.8.27 · H2 (embedded) · Flyway 10

INFO

LLM Models

llama3.1:8b — reasoning, planning, docs
qwen2.5-coder:7b — code generation only

INFO

Build

Maven · 94 unit tests · MiniJUnit runner · No Maven
Central needed (jars compiled from GitHub source)

INFO

Deployment

Docker multi-stage build · docker-compose with Ollama
sidecar · ollama-pull init service

docker-compose Topology

```
services:
  ollama:
    # Ollama model server
    image: ollama/ollama
    volumes: [ollama_data:/root/.ollama]
    ports: [11434:11434]

  ollama-pull:
    # Init container: pulls models once
    image: ollama/ollama
    depends_on: [ollama]
    command: |
      ollama pull llama3.1:8b
      ollama pull qwen2.5-coder:7b

  app:
    # Spring Boot application
    build: .
    depends_on: [ollama-pull]
    ports: [8080:8080]
    environment:
      SPRING_AI_OLLAMA_BASE_URL: http://ollama:11434
```

Database Schema

```
-- V1: URL shortener tables
CREATE TABLE short_urls (
  code          VARCHAR(20) PRIMARY KEY,
  original_url  TEXT NOT NULL,
  created_at    TIMESTAMP,
  active        BOOLEAN DEFAULT TRUE
);

CREATE TABLE click_events (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  code          VARCHAR(20) REFERENCES short_urls,
  clicked_at    TIMESTAMP
);

-- V3: SDLC run persistence (paused states survive restart)
CREATE TABLE paused_runs (
  run_id       VARCHAR(36) NOT NULL PRIMARY KEY,
  state_json   TEXT        NOT NULL,
  gate         VARCHAR(100),
  paused_at    TIMESTAMP   NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Scenario Types

GREENFIELD

New service from scratch

Full 8-agent run. Codebase analysis finds no existing code. Implementation generates complete service.

BROWNFIELD

Enhancement to existing code

CodebaseAnalysis identifies impacted modules. ImplementationAgent generates targeted patch code rather than a full service.

AMBIGUOUS

Unclear requirement

RequirementsAgent detects ambiguity. Graph pauses at a clarification gate before proceeding. Client must POST clarification to resume.